

Modern LLM inference executes each decoder layer as many separate GPU kernels. Fleet explores a different execution model: keeping GPU workers resident and coordinating dependent operations through an on-device task graph, reducing host involvement and kernel-launch overhead. The task is to design and implement as much of a Fleet-style batch-1 decode path as possible for DeepSeek-Coder-V2-Lite-Base on one AMD MI300X, progressing from model and tensor-flow analysis to one validated decoder layer, multiple persistent layers, and, if possible, end-to-end token generation. Full completion is not expected; evaluation will focus on the correctness and depth of the implementation, the milestone reached, and the ability to identify and explain remaining limitations.

Recommended approach

First deliverable: Submit a technical design describing the model execution flow, proposed Fleet task graph, synchronization strategy, memory plan, correctness methodology, implementation milestones, expected performance, and work that can be completed locally before using the GPU.

Implementation: MI300X access will be provided, but no prepared software environment will be supplied. Set up the development environment, install the required dependencies, download and configure the model, establish a correctness reference, and implement the Fleet execution path. Work that does not require GPU execution may be completed locally. GPU implementation should progress from a validated operator or decoder layer to multi-layer persistent execution and, if feasible, end-to-end decode.

Model: DeepSeek-Coder-V2-Lite-Base

Hardware: One AMD MI300X

Execution: Batch-1 autoregressive decode

Precision: BF16 initially

Input context: 1,024 tokens

Output: 32 greedily decoded tokens

No tensor parallelism

No continuous batching

No speculative decoding

Required single-layer milestone: The first complete decoder layer validated through the Fleet execution path must be an MoE layer with layer index 1 or higher. A layer-0-only implementation does not satisfy this milestone because layer 0 uses a dense MLP and does not exercise expert routing or expert execution.

Prefill scope: Implementing or optimizing prefill is out of scope. A stock reference implementation may process the 1,024-token prompt and initialize the KV cache; only batch-1 decode is evaluated as part of the Fleet path. The design specification must define the interface between reference prefill and Fleet decode, including the KV-cache layout. If the reference prefill produces a cache representation incompatible with the proposed MLA decode path, the implementation must convert it or choose a compatible representation, clearly document that decision, and exclude any one-time conversion from measured decode latency.

Metrics: Report correctness error, Fleet-native operations, remaining fallbacks, number of consecutive layers completed, GPU launches, median and P95 latency, memory traffic, achieved bandwidth, occupancy and GPU resource usage. If end-to-end decode is reached, also report time per output token and tokens per second. No separate baseline implementation or baseline profiling is required.

Final submission: Provide all source code, environment setup scripts, build and run instructions, correctness evidence at every completed boundary, profiling commands and results, exact milestone reached, remaining fallbacks, known failures, and recommended next steps. All results must be reproducible on the provided MI300X machine.

Time limit: Five days.

Reference:

<https://arxiv.org/html/2604.15379v1>

<https://github.com/ROCm/fleet-chiplet-megakernel>